MASTER'S MODULE · AGENTIC AI · WINTER SEMESTER

Orchestration: ReAct, Plan-and-Execute, DAGs

Who decides the next step — your code or the model?

Unit 05 · Part 1 of 2 · Session 06 · Week 6
Prof. Dr. Hauke Schramm · Department of Computer Science · Kiel University

Today's agenda

1. **What orchestration means** control flow as a design decision
2. **ReAct** interleaved thought, action, observation
3. **Plan-and-Execute** upfront plans, cheap execution, replanning
4. **Workflows & DAGs** code-owned control flow, five patterns
5. **Choosing a regime** predictability, flexibility, cost, debuggability

Where we are

Unit 05, part 1 — from components to control flow

Last week (S05) — we chose models per role: cheap workers, reasoning planners and verifiers

This week — how the steps connect: who sequences the calls, branches, and loops

The shift — from picking components to wiring the control flow between them

Next week (S07) — frameworks vs plain code — how to implement today's patterns

GUIDING QUESTION

When should your code decide the next step — and when should the model?

1

What orchestration means

Control flow as a design decision

Orchestration, defined

Orchestration — the control layer: which step runs next, with what input

Computation vs control — LLM calls and tools do the work; orchestration sequences it

The design question — is the next step chosen by your code or by the model?

A spectrum — fixed chains, branching workflows, planned runs, fully dynamic loops

Recap (S01/S02) — Anthropic: workflows follow code-defined paths; agents direct their own

THIS UNIT

S06: the control-flow patterns themselves. S07: implementing them — framework or plain code.

The one question

Who decides the next step — your code or the model?

Every pattern in this session is one answer. Code-owned control maximises predictability; model-owned control maximises flexibility. Everything else — cost, latency, debuggability — follows from where you sit on that axis (cf. Anthropic, 2024).

2

ReAct

Interleaved reasoning and acting (Yao et al., 2023)

ReAct: reason, act, observe

Yao et al., ICLR 2023 — the canonical dynamic pattern

Core idea — interleave free-text reasoning with tool actions in one loop

Thought — the model reflects: what do I know, what is missing, what next?

Action — a tool call — search, fetch, finish — emitted as text

Observation — the tool result, appended to context before the next thought

Then a prompt trick — few-shot traces taught the format; no API support existed yet

LINEAGE

Your S02 loop with native tool calling is ReAct-inspired: related, but without the paper's explicit reasoning interleave it is not the same procedure.

A ReAct trace

Synthetic, hand-written trace (not a real run): an agent with search and fetch tools verifies one claim

react_trace_synthetic.txt

```
Question: When do the EU AI Act obligations for
  general-purpose AI (GPAI) models apply?
Thought 1: I need the official timeline, not a news
  summary. Search for a Commission page.
Action 1: search("EU AI Act GPAI obligations date")
Observation 1: [news article 2024-03; European
  Commission: Guidelines for GPAI providers]
Thought 2: The Commission page is primary. Fetch it.
Action 2: fetch("digital-strategy.ec.europa.eu/en/
  policies/guidelines-gpai-providers")
Observation 2: "obligations apply from 2 August 2025;
  models placed on the market before that date
  have until 2 August 2027 to comply ..."
Thought 3: Confirmed by a primary source. Done.
Action 3: finish("GPAI duties from 2 Aug 2025;
  pre-existing models: transition to 2 Aug 2027")
```

Why interleaving wins

Yao et al.'s ablations: each mode alone fails in its own way

Reason-only (CoT)

No world access: the chain runs on parametric memory alone, hallucinates facts confidently, and propagates early errors unchecked.

Act-only

No explicit reasoning trace between calls: the agent still reacts to observations, but goal decomposition and progress tracking stay implicit, harder to inspect or steer.

ReAct: interleaved

Thoughts create, track and update plans; observations ground thoughts in the world. Errors surface early — and trigger recovery.

ReAct's failure modes

Grounded and adaptive — with failure modes you can observe in a trace

Myopia — a possible failure, not a definition: thoughts lose the plan and chase the last observation

Looping — re-issuing a failed search with minor rewording, burning budget

Derailment — one misleading observation pulls the whole trajectory off course

Context growth — every thought and observation accumulates; long runs hit the window

Mitigations — step caps, loop detection, compaction (S08) — or: plan first

LAB LINK

Lab 06: build ReAct from scratch (Thought/Action/Observation parsed from text), trace every step on an offline corpus, hit the loop trap, write a loop detector (R2).

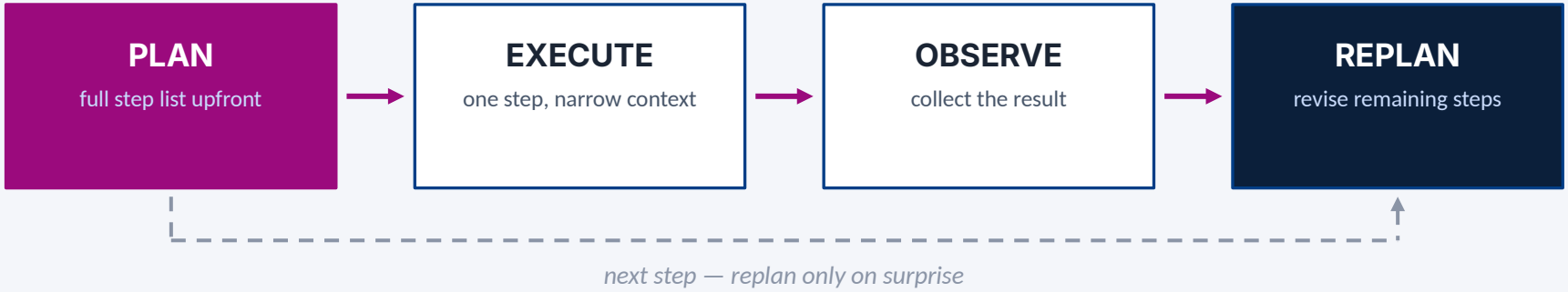
3

Plan-and-Execute

Plan first, act cheaply, replan on surprise

The Plan-and-Execute loop

Decisions are front-loaded; the executor's decisions are narrower, not absent



- The planner sees the goal; the executor sees one step plus the goal's constraints and the results it depends on.
- Replanning is an explicit, observable event — not silent drift. Lab 06 (Part D) logs each replan as an event.

Mechanics and cost profile

Plan as artifact — an explicit step list you can log, inspect, approve, test

Executor — runs steps with narrow context — often a cheaper model (S05)

Replan triggers — step failure, surprising observation, budget threshold

Cost shape — few heavy planner calls bracket many light executor calls

Weakness — plans go stale; volatile environments force constant replanning

Measure it — Lab 06: plan-first vs ReAct on the same tasks and budget; log replan counts

EVIDENCE, SCOPED

Plan-and-Solve (Wang et al., 2023) supports explicit plan-first prompting. It is not a direct evaluation of this tool-using planner-executor architecture.

ReAct vs Plan-and-Execute

Two dynamic regimes, different decision rhythms

ReAct (interleaved)

- a decision after every observation
- adapts instantly to surprises
- full history in context — grows fast
- cost emerges step by step at run time
- no separate initial planning phase

Plan-and-Execute

- decisions front-loaded into the plan
- adapts only at replan points
- executor context stays small
- cost estimable once the plan exists
- upfront latency before the first action

4

Static workflows and DAGs

When code owns the control flow

The static end: DAG orchestration

DAG — directed acyclic graph: nodes are steps, edges are data dependencies

Execution — topological order; independent branches can run in parallel if resources, rate limits and sync allow

Construction — at design time, or at run time by an orchestrator; the LLM fills node content

Termination — a finite DAG with terminating nodes and bounded retries terminates; a hanging node still hangs

Iteration — needs a cyclic graph or finite unrolling; graph frameworks allow cycles (S07)

Heritage — data-engineering orchestrators (Airflow); LangGraph and others for agents (S07)

WHAT ACYCLICITY BUYS

It rules out unbounded loops in the control flow, not a node that never returns. A ReAct loop is a cyclic graph: that cycle is its power and its risk.

Five workflow patterns

Anthropic, Building Effective Agents (2024) — code-defined control flow

Prompt chaining

A fixed sequence of calls, each consuming the previous output; programmatic gates between steps catch bad intermediates early.

Routing

A classifier step sends each input down one of several specialised branches — easy cases handled cheaply, hard cases escalated.

Parallelization

Fan out, fan in. Sectioning: independent subtasks run concurrently. Voting: the same task sampled N times, then aggregated.

Orchestrator-workers

A central LLM decomposes the task at run time, delegates subtasks to workers, merges results — the bridge toward agents.

Evaluator-optimizer

A generator drafts; an evaluator critiques against criteria; repeat until pass or a fixed round cap. A bounded cycle, not a DAG.

Beyond: the agent

When subtasks cannot be predicted in advance, drop the script: the model directs its own loop — the S02 pattern.

Fan-out and fan-in: orchestrator-workers

Example: a report-writing workflow that splits a topic into subtopics



- Sectioning cuts wall-clock time only if workers get their own capacity; each starts with a clean, small context.
- Voting variant: three evaluators judge one source; majority decides (cf. S05 self-consistency).
- Fan-in is the hard part: merging drafts needs its own synthesis call. Lab 06 Part F: a two-branch DAG with one fan-in node.

5

Choosing a control regime

Predictability versus flexibility

Three regimes, side by side

Axis	Static DAG	Plan-and-Execute	ReAct (dynamic)
Next step chosen by	code, at design time	planner, at run start	model, at every step
Predictability	highest: transitions fixed in code	medium: plan visible upfront	lowest: path emerges
Flexibility	only coded branches	replanning on surprise	adapts to every observation
Debuggability	unit-test each node	inspect the plan artifact	read the full trace
Cost	bounded by structure; varies with inputs	planner-heavy, executor cheap	unbounded without caps
Best for	stable, repeated processes	long tasks, stable subgoals	open-ended, unpredictable tasks

No regime dominates — the task's predictability decides, not fashion.

Debuggability and cost

Static workflows — code defines allowed transitions; realised paths and costs may still vary

Dynamic agents — paths differ per run; model failures are statistical, infrastructure bugs stay reproducible

Tracing — dynamic control demands full traces — every thought, call, observation (S11)

Caps — dynamic loops need budgets: max steps, tokens, money, wall-clock time

Anthropic's rule — use the simplest pattern that works; add autonomy only when measured

RULE OF THUMB

Predictable subtask: wire it in code. Unpredictable task shape: let the model decide — inside guardrails.

Lab 06's claim-verification task, two ways

Same offline corpus (data/corpus.json) and tools, opposite ends of the control axis

As a static DAG (Part F)

- fixed nodes: search, fetch, extract, draft
- two parallel branches: timeline, penalties
- optional: one fixed revision node after draft, no loop
- transitions fixed; node contents vary per run
- testable per node; cost bounded by structure

As a ReAct agent (Part C)

- one loop; the model picks each action
- follows leads a fixed graph would miss
- falls into the loop trap without a detector
- path, steps and tokens vary per run
- needs a step cap, tracing, loop detector (R2)

TAKEAWAYS

What you should remember

- **Orchestration —** the control layer — who picks the next step
- **ReAct —** interleaved thought, action, observation: grounded, adaptive; looping and myopia are failure modes
- **Plan-and-Execute —** explicit plan artifact, narrower executor, observable replanning
- **Static DAGs —** code owns transitions: parallelisable, testable; terminates if nodes and retries do
- **Five patterns —** chaining, routing, parallelization, orchestrator-workers, evaluator-optimizer
- **Choosing —** predictability vs flexibility — buy autonomy only where needed

NEXT SESSION Frameworks vs plain code — the implementation decision.

Reading & references

- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023. arXiv:2210.03629.
  The primary text: interleaved reasoning and acting, with ablations against reason-only and act-only.
- Anthropic (2024). Building Effective Agents. Engineering blog.
  The workflow pattern catalogue: chaining, routing, parallelization, orchestrator-workers, evaluator-optimizer.
- Wang, L. et al. (2023). Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models. ACL 2023. arXiv:2305.04091.
  Plan-first prompting evidence — the prompting ancestor of plan-and-execute.
- Wei, J. et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. arXiv:2201.11903.
- LangChain (2023). Plan-and-Execute Agents. Engineering blog.
- LangGraph documentation, LangGraph 1.2 (2026). docs.langchain.com/oss/python/langgraph.
  Graph-based agent orchestration in practice — preview of S07.
- Wang, Lei et al. (2024). A Survey on LLM-Based Autonomous Agents. arXiv:2308.11432.
- European Commission (2025). Guidelines for providers of GPAI models. digital-strategy.ec.europa.eu/en/policies/guidelines-gpai-providers (checked 2026-10-01). Primary source for slide 9.